

INF01151 – Sistemas Operacionais II

Aula 04 - Exclusão mútua (implementação)

Prof. Alberto Egon Schaeffer Filho

O problema da seção crítica ...

- Seção crítica **Processo deve executar a seção crítica o mais rapidamente possível!**
 - Porção de código que não pode ser executado por dois ou mais processos simultaneamente sob pena de inconsistência de dados
- Necessário garantir acesso exclusivo de um processo à seção crítica
 - Se um processo está executando instruções da seção crítica, outro processo é impedido de entrar até que o primeira saia
 - É o que se denomina de **exclusão mútua**
 - Garantir acesso **exclusivo!**
 - Resulta em uma **serialização** no acesso
- Primitivas para exclusão mútua:
 - **Enter_EM**
 - **Exit_EM**



Como implementar?

- Primitiva genérica: `<await (B); S>`
 - Exclusão mútua e sincronização condicional
- Como implementar o operador exclusão mútua (“<” e “>”)?
 - Sem suporte de hardware, ou seja, em software puro
 - Algoritmo de Dekker
 - Algoritmo de Peterson
 - Algoritmo de Lamport
 - Com suporte de hardware
 - Baseado em habilitação/desabilitação de interrupções
 - Instruções atômicas (test-and-set e swap)

Propriedades para primitivas de exclusão mútua

(1) Exclusão mútua

- Garantir que no máximo um processo esteja executando dentro da seção crítica em um determinado instante

(2) Progressão

- Um processo fora da seção crítica não pode impedir outro processo de entrar

(3) Espera limitada

- Um processo não deve ser indefinidamente impedido de entrar na seção crítica

(4) Não deve fazer suposições sobre a velocidade relativa da execução dos processos, nem velocidade de processadores

Processo ou *thread*

Considerações gerais

- Para simplificar a análise:
 - Duas unidades de execução (threads ou processos)
 - Algoritmos podem ser adaptados para n threads ou processos
- Máquinas com um só processador (e um só core)
- Programas tem três partes:
 - Instruções críticas (modificam dados compartilhados)
 - Instruções não críticas (não atualizam dados compartilhados)
 - Primitivas para exclusão mútua (**Enter_EM** e **Exit_EM**)



Abordagem algorítmica

- Abordagem algorítmica é caracterizada por:
 - Não empregar serviços providos pelo núcleo (kernel) para bloquear, sinalizar ou retardar a execução de um processo
 - Não empregar instruções especiais do processador
- Vantagem:
 - É uma solução independente de sistema operacional e de plataforma de execução
- Desvantagem:
 - Emprego de espera ativa (busy waiting)
 - Complexidade de programação

Algoritmo de Dekker (versão I)

```
Var      turn: 1..2
Begin
    turn := 1;

Parbegin
    repeat
        while (turn = 2) do {nothing};
        { seção crítica};
        turn := 2;
        {seção não crítica}
    until (forever);
Parenend ;
End.
```

Problema?

Viola propriedade da progressão

```
repeat
    while (turn = 1) do {nothing};
    { seção crítica};
    turn := 1;
    {seção não crítica}
until (forever);
```

Algoritmo de Dekker (versão II)

```
Var      p1Inside, p2Inside: 0..1;
Begin
    p1Inside := 0;
    p2Inside := 0;
Parbegin
    repeat
        while (p2Inside = 1) do {nothing};
        p1Inside := 1;

        {seção crítica}

        p1Inside := 0;

        {seção não crítica}
    until (forever);
Parend;
End.
```

p1Inside e p2Inside são flags para indicar se o processo P_i está na região crítica. Antes de entrar na SC um processo verifica se a mesma livre (se o outro processo não está dentro dela).

```
repeat
    while (p1Inside = 1) do {nothing};
    p2Inside := 1;

    {seção crítica}

    p2Inside := 0;

    {seção não crítica}
until (forever);
```

Problema?

Viola regra da exclusão mútua

Algoritmo de Dekker (versão III)

```
Var      p1WantsToEnter,
         p2WantsToEnter: 0..1;
Begin
    p1WantsToEnter := 0;
    p2WantsToEnter := 0;
Parbegin
    repeat
        p1WantsToEnter := 1;
        while (p2WantsToEnter = 1)
            do {nothing};

        {seção crítica}

        p1WantsToEnter := 0;

        {seção não crítica}
    until (forever);
Parend;
End.
```

Problema?

Viola regra da espera limitada
(*livelock*)

```
repeat
    p2WantsToEnter := 1;
    while (p1WantsToEnter = 1)
        do {nothing};

    {seção crítica}

    p2WantsToEnter := 0;

    {seção não crítica}
until (forever);
```

Algoritmo de Dekker (versão IV)

```
Var      p1WantsToEnter,  
         p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;  
Parbegin  
    repeat  
        p1WantsToEnter := 1;  
        while (p2WantsToEnter = 1) do {  
            p1WantsToEnter := 0;  
            timewait(rand);  
            p1WantsToEnter := 1;  
        }  
        {seção crítica}  
        p1WantsToEnter := 0;  
        {seção não crítica}  
    until (forever);  
Parenend;  
end.
```

Problema?

Um processo pode sempre dar preferência para o outro e nenhum deles ingressar na SC (problema do adiamento indefinido)

```
repeat  
    p2WantsToEnter := 1;  
    while (p1WantsToEnter = 1) do {  
        p2WantsToEnter := 0;  
        timewait(rand);  
        p2WantsToEnter := 1;  
    }  
    {seção crítica}  
    p2WantsToEnter := 0;  
    {seção não crítica}  
until (forever);
```

Algoritmo de Dekker (versão V)

```
Var turn: 1..2;  
    p1WantsToEnter, p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;  
    turn := 1;  
Parbegin  
    repeat  
        p1WantsToEnter := 1;  
        while (p2WantsToEnter = 1) do  
            if (turn = 2) then {  
                p1WantsToEnter := 0;  
                while (turn = 2) do {nothing};  
                p1WantsToEnter := 1;  
            }  
        {seção crítica}  
        turn := 2; p1WantsToEnter := 0;  
    {seção não crítica}  
    until (forever);  
Paren.  
End.
```

Problema?

Dessa vez não... funcionamento está correto! Mas e para n processos?

```
repeat  
    p2WantsToEnter := 1;  
    while (p1WantsToEnter = 1) do  
        if (turn = 1) then {  
            p2WantsToEnter := 0;  
            while (turn = 1) do {nothing};  
            p2WantsToEnter := 1;  
        }  
    {seção crítica}  
    turn := 1; p2WantsToEnter := 0;  
    {seção não crítica}  
until (forever);
```

Algoritmo de Peterson (1981)

```
Var turn: 1..2;  
    p1WantsToEnter, p2WantsToEnter: 0..1;  
Begin  
    p1WantsToEnter := 0;  
    p2WantsToEnter := 0;  
Parbegin  
    repeat  
        p1WantsToEnter := 1;  
        turn := 2;  
        while (p2WantsToEnter = 1 and turn = 2)  
            do {nothing};  
        {seção crítica}  
        p1WantsToEnter := 0;  
        {seção não crítica}  
    until (forever);  
Parend.  
End.
```

Problema?

Também está correto... mas e para n processos?

```
repeat  
    p2WantsToEnter := 1;  
    turn := 1;  
    while (p1WantsToEnter = 1 and turn = 1)  
        do {nothing};  
    {seção crítica}  
    p2WantsToEnter := 0;  
    {seção não crítica}  
until (forever);
```

Algoritmos para n processos

- Algoritmo de Dijkstra (1965)
 - Possibilidade de adiamento indefinido
- Algoritmo de Knuth (1965)
 - Resolvía adiamento indefinido, mas permite longos atrasos
- Algoritmo de Eisenberg e McGuire (1972)
 - Entrada na seção crítica no máximo após $n-1$ tentativas
- Algoritmo de Lamport (1974)
 - Utiliza sistema do tipo “pegue uma ficha”, por isso chamado de *“algoritmo da padaria”*

Algoritmo da Padaria (Lamport, 1974)

```
const n;  
Var choosing = array[0..n-1] of boolean;  
    ticket = array[0..n-1] of integer;  
Begin  
  for j=0 to n-1 do {  
    choosing[j] := false;  
    ticket[j] := 0;  
  }
```

Parbegin Process **Pi**:

repeat

```
  choosing[i] = true;  
  ticket[i] := max(ticket[0]...ticket[n-1]) + 1;  
  choosing[i] := false;  
  for j = 0 to n-1 do {  
    while (choosing[j]) do {nothing};  
    while (ticket[j] ≠ 0 and (ticket[j], j) < (ticket[i], i)) do {nothing};  
  }
```

{Seção crítica}

```
  ticket[i] := 0;
```

{Seção não crítica}

until (forever);

Parend.

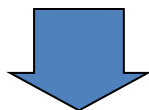
End.

Tie break (relação precedência)

$(\text{ticket}[j], j) < (\text{ticket}[i], i) \rightarrow$
 $(\text{ticket}[j] < \text{ticket}[i])$ or
 $(\text{ticket}[j] = \text{ticket}[i] \text{ and } j < i)$

Problemas com soluções em software

- Software puro: algoritmos de Dekker, Peterson e Lamport
 - Principais problemas?
 - **Complexidade**
 - **Problema de desempenho**
 - **Espera ativa** (busy wait)
 - **Falta de clareza** entre o que é **lógica de controle** de acesso a seção crítica e o que é da **lógica do programa**



Solução

Contar com o auxílio de mecanismos de hardware (projeto do processador)

Mecanismos de hardware

- Habilitação e desabilitação de interrupções
- Instruções específicas a serem usadas como base para se implementar primitivas de exclusão mútua:
 - Test and set
 - Swap



Unidades de execução: *thread* ou processos
(discurso válido para ambos!)

Desabilitando interrupções

- Preempção permite que múltiplos processos acessem dados compartilhados assincronamente
 - Preempção geralmente devida a ocorrência de **interrupções de tempo** (sinalizam expiração do quantum)
- Desabilitação de interrupções! Problemas?
 - Poder demais para um usuário! Se processo entrar em loop, não irá sair
 - Não funciona em máquinas multiprocessadoras, pois dois processos ainda podem executar ao mesmo tempo em processadores diferentes



Instruções especiais de hardware

- Desabilitar interrupções raramente é uma solução prática...
- Projeto de processadores levam em conta que eles serão usados em ambientes de programação concorrente
- Instruções assembly para leitura e escrita de posições de memória de forma atômica (indivisível)
 - **Test and Set: test-and-set (a, b)**
 - Lê o valor de uma posição de memória e coloca nela um valor não zero
 - **SWAP: swap (a, b)**
 - Permutação de valores

Instrução test-and-set

- Problema comum de sincronização: preempção de thread após ter lido valor de memória, mas antes de ter escrito novo valor
 - Verificar exemplos de exclusão mútua em software: thread precisa ler um valor para determinar que nenhum outro está na seção crítica, e atualizar uma variável de controle
 - Porém thread poderia sofrer preempção entre as duas operações!!
- Instrução test-and-set: habilita leitura de um valor e escrita de um novo valor atomicamente
 - Instrução em si não impõe exclusão mútua, mas pode ser usada por programadores para facilitar soluções

test-and-set(*a*,*b*)

- Lê valor de ***b*** (true|false). Copia valor para ***a*** e coloca o valor de ***b*** em true

test-and-set(*p1MustWait*, *occupied*)

Instrução test-and-set

- Problema comum de sincronização: preempção de thread após ter lido valor de memória, mas antes de ter escrito novo valor
 - Verificar exemplos de exclusão mútua em software: thread precisa ler um valor para determinar que nenhum outro thread pode escrever em uma variável de controle
 - Porém thread poderia sofrer preempção
- Instrução test-and-set: habilita uma thread a escrever um novo valor atomicamente
 - Instrução em si não impõe exclusão mútua, mas permite que programadores facilitem soluções

```
while (p1MustWait)
{
    test-and-set(p1MustWait, occupied);
}
```

// código da seção crítica

```
p1MustWait = true;
occupied = false;
```

test-and-set(a, b)

- Lê valor de **b** (true|false). Copia valor p

test-and-set(p1MustWait, occupied)

Instrução swap

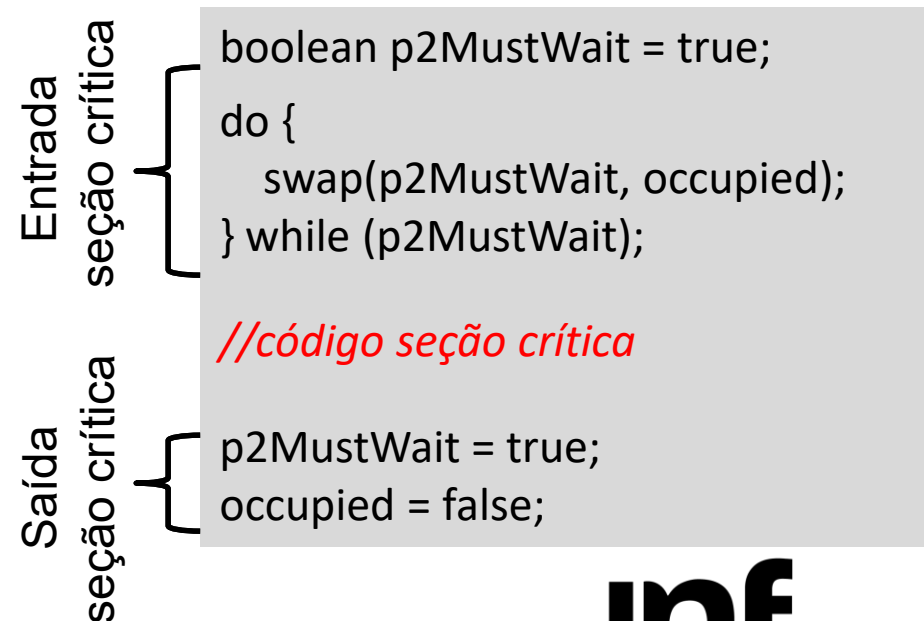
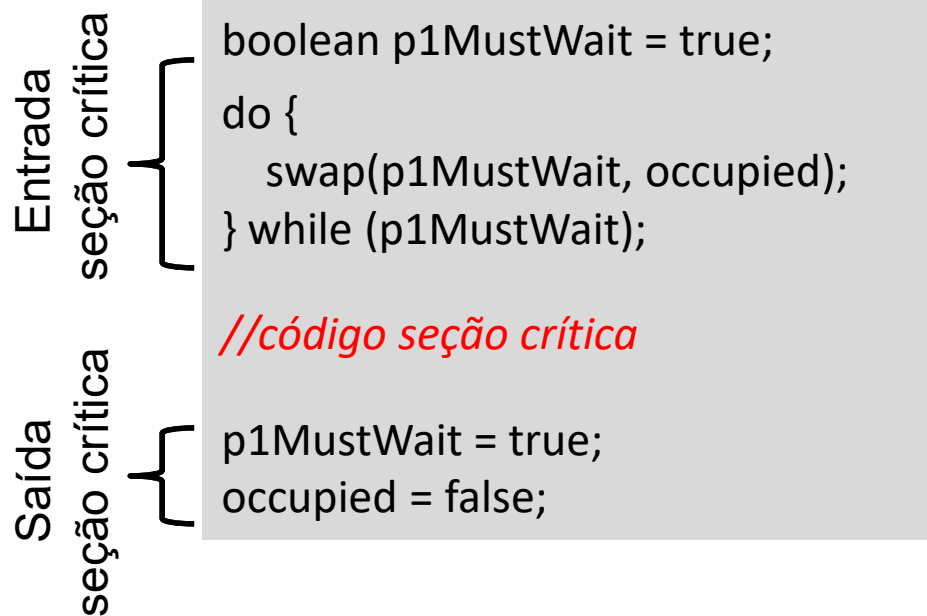
- Instrução Swap:

- Pode fornecer funcionalidade similar à instrução test-and-set

swap (a,b)

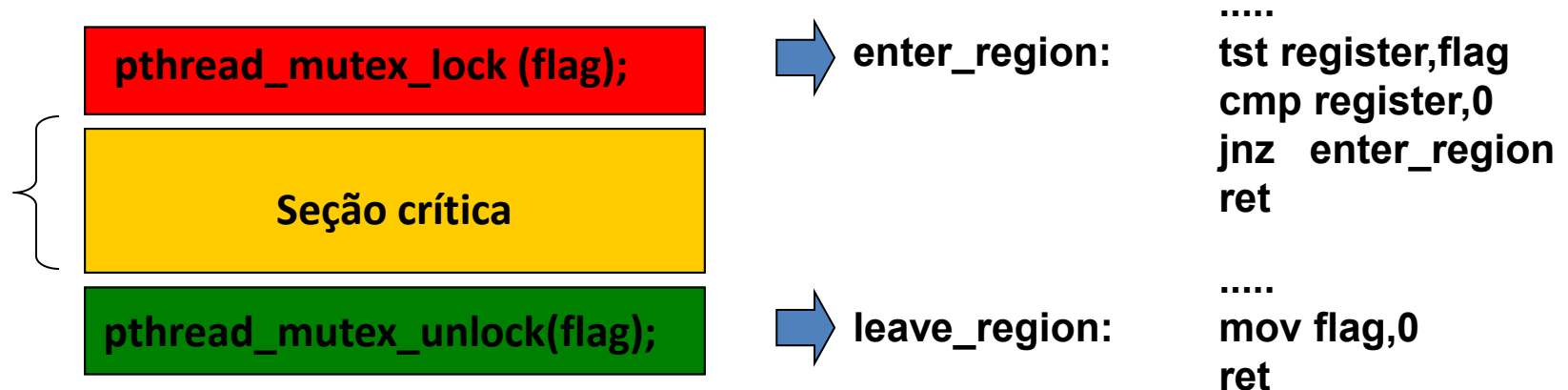
- Instrução carrega valor de **b** (true|false) em registrador **temporário**.
Copia valor de **a** em **b**. Copia valor do registrador **temporário** para **a**

```
tem = b;  
b = a;  
a = tem;
```



Primitivas *lock* e *unlock*

- Instruções especiais podem ser usadas para implementar primitivas de mais alto nível
 - Solução freqüentemente referenciada como ***mutex*** (***pthread_mutex_t***)



Problemas com a solução *lock/unlock*

- Uso correto das primitivas *lock* e *unlock*
 - Programador deve estar ciente (para não dizer “ser competente!”) de que é necessário liberar o acesso a seção crítica (*unlock*)
 - Esquecimento gera, normalmente, *deadlock*
- Espera ativa (*busy wait*)
- Inversão de prioridade

Problema e solução para a espera ativa

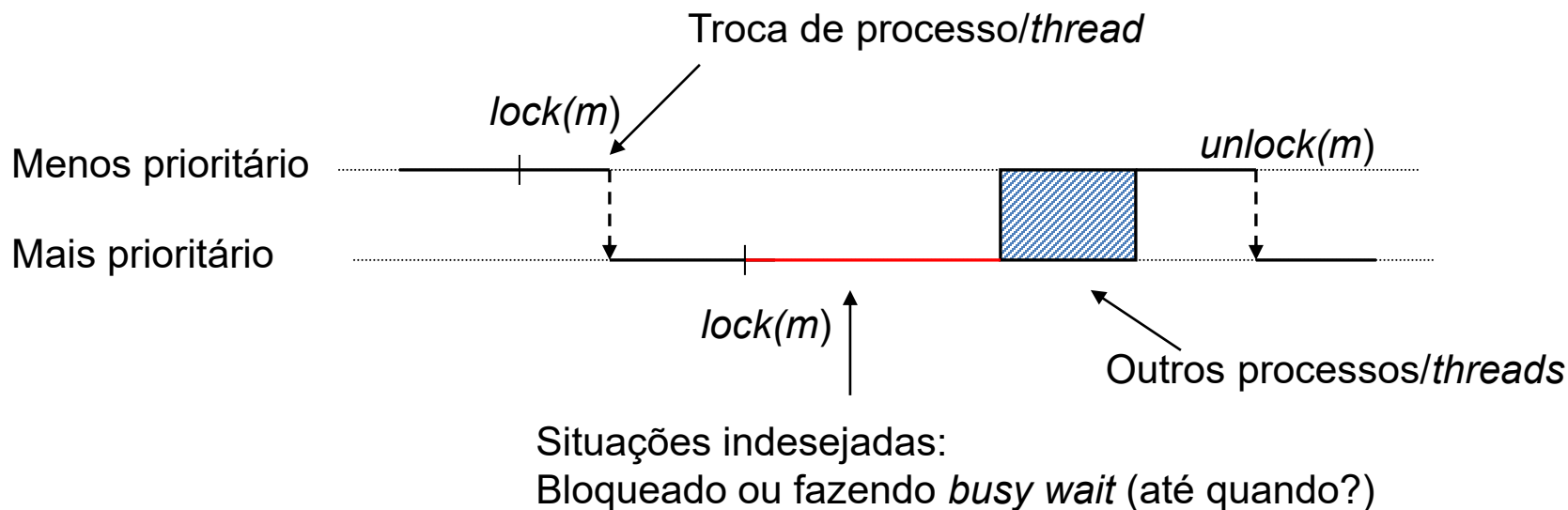
- Espera ativa ou *busy wait* (*spin lock*)
 - Sem sentido, pois para alguém liberar o recurso deve executar, mas para executar tem que usar a CPU
 - Argumento válido para monoprocessadores (*monocore*)
- Solução alternativa: bloquear o processo (*thread*)
 - Baseado em duas novas primitivas mais elaboradas
 - *sleep*: Bloqueia um processo a espera de uma sinalização
 - `pthread_cond_wait`
 - *wakeup*: Sinaliza um processo
 - `pthread_cond_signal`

Mas, espera ativa nem sempre é um problema...

- Em multiprocessadores (multicore) o *busy wait* pode ser interessante
 - Há outro “processador” para liberar o recurso
 - Custo de chaveamento **pode** não compensar (relação com o tamanho da seção crítica)
 - ✓ Porém viola a regra sobre “fazer suposições sobre processadores”
- Possibilidade: primitiva *trylock*
 - Faz *busy wait* por um período de tempo, se durante esse período o *mutex* não tiver sido liberado bloqueia (*sleep*)
 - Programador escolhe entre *trylock* e *lock*
 - Problema é dimensionar o período de tempo
 - Compromisso entre o tamanho da seção crítica, tempo de chaveamento de contexto e tempo estimado de espera

Problema: Inversão de prioridades

- Uma *thread* de mais baixa prioridade impedir a progressão de uma *thread* de mais alta prioridade



Soluções para inversão de prioridade

- Teto para prioridade (priority ceiling)
 - Aumento da prioridade de uma thread para um valor pré-determinado sempre que ela adquirir um mutex (lock)
 - Teto deve ser maior que a prioridade da maior thread
 - Na liberação (unlock) a thread retorna a sua prioridade original
- Herança (priority inheritance)
 - A prioridade da thread que detém o mutex é elevada ao nível da thread mais prioritária que solicita o recurso
 - Elevação da prioridade só ocorre se houver risco de conflito
 - Na liberação (unlock) a thread retorna a sua prioridade original

Leituras adicionais

- Deitel, Deitel e Choffnes, Sistemas Operacionais, 3a edição, Pearson Prentice Hall
 - Capítulo 5 (seções 5.1 a 5.5)